Assignment 1: VGG6 on CIFAR-10

Exploring activations, optimizers and training hyperparameters

Name / Roll No.	Rajalakshmi S / CS25M528
GitHub repository	https://github.com/srajiniranjith-collab/vgg6-cifar10
W&B project	https://wandb.ai/cs25m528-iit-madras/vgg6-cifar10
W&B sweep: activation	https://wandb.ai/cs25m528-iit-madras/vgg6-cifar10/sweeps/qhgtddfc
W&B sweep: batch size x lr	https://wandb.ai/cs25m528-iit-madras/vgg6-cifar10/sweeps/cs5i18sl
W&B sweep: optimizer x lr	https://wandb.ai/cs25m528-iit-madras/vgg6-cifar10/sweeps/eo3eytmb
W&B sweep: architecture x label smoothing	https://wandb.ai/cs25m528-iit-madras/vgg6-cifar10/sweeps/xa20d8e2
W&B sweep: epochs (run twice)	https://wandb.ai/cs25m528-iit-madras/vgg6-cifar10/sweeps/inrr1jfj https://wandb.ai/cs25m528-iit-madras/vgg6-cifar10/sweeps/0az70lwc

Setup used throughout. The default network is the template VGG6 (4 conv layers with BatchNorm and ReLU, 2 max-pools, global average pooling, one linear layer; **262,218** parameters); Section 2d compares it with a deeper and a wider variant. The 50,000 CIFAR-10 training images are split into **45,000 train / 5,000 validation** with a fixed split; the official 10,000 test images are used only to report final accuracy. Model selection always uses validation accuracy. All runs use seed 42 and a constant learning rate (no scheduler). 'Best val acc' is the maximum validation accuracy over epochs, and 'test acc' is measured at that same epoch.

Question 1. Training baseline

(a) Data preparation

Split	Transforms (in order)
Train (45k)	RandomCrop(32, padding=4); RandomHorizontalFlip(p=0.5); ToTensor; Normalize(mean = (0.4914, 0.4822, 0.4465), std = (0.2023, 0.1994, 0.2010)); Cutout(1 hole, 16x16 px)
Validation (5k) and Test (10k)	ToTensor; Normalize(same mean / std). No augmentation.

Normalisation uses the per-channel CIFAR-10 training-set statistics. Random crop with padding and horizontal flips provide translation and mirror invariance; Cutout (applied after normalisation, so a masked pixel equals the dataset mean colour) acts as a regulariser by forcing the network to use several regions of the image. The validation split is taken from the training images *without* augmentation so that it measures generalisation on clean images.

(b) Chosen configuration

Among all configurations trained (activation, optimizer, batch size, learning rate, epoch, architecture and label-smoothing runs), the one with the best validation and test accuracy is a six-weight-layer network (5 conv + 1 linear; see Section 2d) trained with label smoothing 0.1:

Architecture	Label smoothing	Activation	Optimizer	Learning rate	Batch size	Epochs	Weight decay	Seed
six_layer	0.1	ReLU	Adam	1e-3	128	30	5e-5	42

(c) Results and curves

Best validation accuracy: 87.22% (epoch 30). Final test top-1 accuracy: 86.29% (weights from the best-validation epoch, which is the last epoch). Figure 1 shows the loss and accuracy curves of this run.

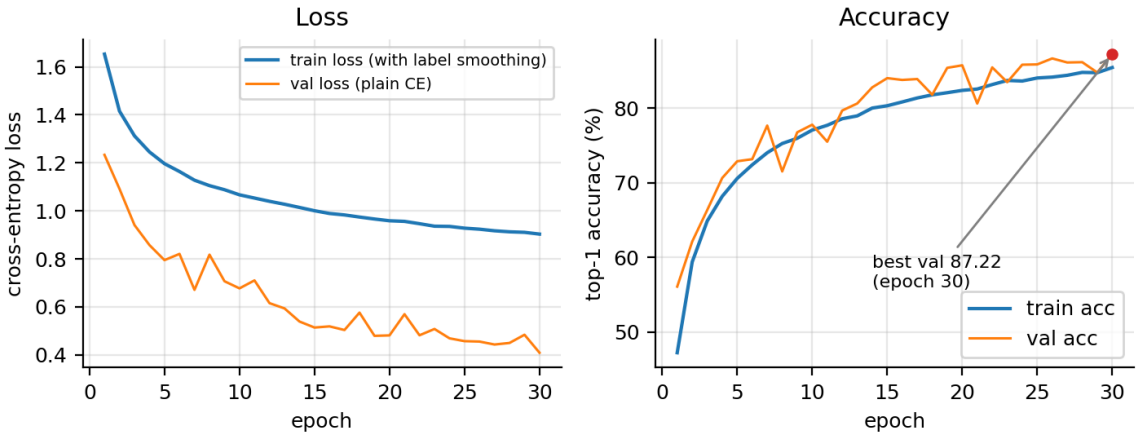


Figure 1. Loss and top-1 accuracy per epoch for the chosen configuration (six_layer, label smoothing 0.1, Adam, lr 1e-3, batch size 128, 30 epochs).

- **Training accuracy sits below validation accuracy** (85.4% at the last epoch vs. 87.2%). This is expected: training accuracy is measured on augmented batches (random crop, flip, Cutout) while validation images are clean.
- **The two loss curves are not directly comparable:** the training loss includes the label-smoothing term (a smoothed target cannot be fitted below a positive floor), so it stays far above the validation loss, which is plain cross-entropy. The validation loss reaches its lowest value (0.409) at the last epoch and there is no sign of overfitting, so the epoch budget is probably still limiting accuracy.
- **Validation accuracy is noisy** because the learning rate is constant: over the last five epochs it was 86.64%, 86.10%, 86.14%, 84.76% and 87.22%, and its mean over the last ten epochs is 85.20%. The reported best (87.22%) is therefore a favourable epoch; the test accuracy of that checkpoint (86.29%) is 0.9 points lower than its validation accuracy, consistent with mild selection bias.
- For comparison, the best template-architecture run (Section 2c) reached 84.66% validation and 83.48% test after 50 epochs.

Question 2. Model performance on different configurations

(a) Activation function

Five activation functions were compared with everything else held fixed: Adam, lr 0.001, batch size 128, BatchNorm (each block is conv, BatchNorm, activation), weight decay 5e-5, 30 epochs, constant learning rate, seed 42, and the same weight initialisation for all. Sweep: https://wandb.ai/cs25m528-iit-madras/vgg6-cifar10/sweeps/qhgtddfc.

Activation	Best val (%)	Best epoch	Test (%)	Final train loss	Final train acc (%)
SiLU	**82.14**	**28**	**80.74**	**0.605**	**79.15**
GELU	81.82	30	80.61	0.572	80.24
ReLU	81.46	26	80.59	0.594	79.39
Tanh	70.46	30	70.07	0.798	72.00
Sigmoid	46.70	12	46.55	0.946	66.44

Table 1. Activation functions (sorted by best validation accuracy; bold = best). 'Final' = last (30th) epoch; training metrics are measured on augmented batches during training.

Results

The five activations fall into three groups. **SiLU (82.14%), GELU (81.82%) and ReLU (81.46%)** are essentially tied: their validation accuracies differ by 0.68 points and their test accuracies (80.74%, 80.61%, 80.59%) by only 0.15 points, well below the roughly 1 point of noise expected from a single seed on a 5,000-image validation set (Section 2c). **Tanh** is clearly worse (70.46% validation, 70.07% test; about 11 points below ReLU), and **Sigmoid** is far worse (46.70% validation, 46.55% test; about 35 points below ReLU).

Smooth activations versus ReLU

GELU reached the lowest final training loss (0.572 vs. 0.594 for ReLU) and SiLU the highest best validation accuracy, but SiLU's training loss (0.605) is slightly higher than ReLU's, so smoothness gave no consistent advantage in this setting. SiLU and GELU keep a small non-zero gradient for negative inputs and have smooth, non-monotonic shapes, which is the usual argument for their small gains over ReLU, but on this small network the gain is within noise. ReLU is also the cheapest: its run took 327 s versus 342 to 344 s for the others (a rough figure, as runtimes on shared hardware vary).

Saturating activations (Tanh and Sigmoid)

Both saturate: Tanh's output is zero-centred but its derivative goes to zero for large inputs, and Sigmoid's derivative is at most 0.25 and its output is always positive. Gradients are therefore attenuated, and learning is slower. This shows up as *underfitting* rather than overfitting: the final training loss is 0.798 for Tanh and 0.946 for Sigmoid (versus 0.57 to 0.61 for the others), and training accuracy is only 72.0% and 66.4% (versus 79 to 80%). BatchNorm before the activation keeps most pre-activations in the sensitive region, which is probably why Tanh still trains reasonably well. Tanh's best epoch was the last one (epoch 30), so it was still improving and the 30-epoch budget probably understates its eventual accuracy.

Sigmoid is also unusual in that its best validation accuracy occurred at epoch 12 and never improved afterwards, while its training accuracy reached 66.4% by the last epoch. Validation images are clean, so a validation accuracy about 20 points below the (augmented) training accuracy is not ordinary overfitting. It suggests that the evaluation-mode network, which uses BatchNorm running statistics, does not match the training-mode network; Sigmoid's non-zero-centred outputs and small variance are a plausible cause. This explanation was not isolated experimentally and should be checked against the Sigmoid run's per-epoch validation curve in W&B.

Caveats

- Each activation was run once (seed 42) at a single learning rate (0.001), chosen as good for ReLU-type networks with Adam. Tanh and Sigmoid might close part of the gap with a different learning rate or more

epochs; this was not tested.

- All runs use BatchNorm, which helps the saturating activations; without it the gap would probably be larger (not tested).
- The ReLU run is identical to the Adam, lr 0.001, batch size 128 runs in Sections 2b and 2c (best 81.46% at epoch 26, test 80.59%), which confirms the seeded pipeline is deterministic.

(b) Optimizer

Six optimizers were each trained at three learning rates (0.001, 0.01, 0.1), giving 18 runs. Everything else was held fixed: batch size 128, ReLU, BatchNorm, weight decay 5e-5, 30 epochs, constant learning rate, seed 42. Momentum was fixed at 0.9 and only affects SGD, Nesterov-SGD and RMSprop (Adam, Adagrad and Nadam have no such parameter in PyTorch). Sweep: https://wandb.ai/cs25m528-iit-madras/vgg6-cifar10/sweeps/eo3eytmb.

Optimizer	lr = 0.001	lr = 0.01	lr = 0.1
SGD (momentum)	64.88 (64.75)	77.62 (76.51)	**82.04 (81.94)**
Nesterov-SGD	65.44 (65.03)	79.14 (79.04)	**82.26 (81.65)**
Adam	**81.46 (80.59)**	76.54 (74.73)	47.50 (47.53)
Adagrad	60.04 (59.55)	75.50 (73.55)	**80.92 (79.40)**
RMSprop	**77.66 (76.84)**	49.28 (49.46)	13.56 (13.97)
Nadam	**80.90 (79.33)**	77.92 (76.51)	52.94 (52.84)

Table 2. Best validation accuracy % (test accuracy % in brackets) for each optimizer and learning rate; bold = best learning rate for that optimizer.

Optimizer	Best lr	Best val (%)	Best epoch	Test (%)	Train loss @30	Val @ epoch 10	First epoch val >= 70%
Nesterov-SGD	0.1	82.26	29	81.65	0.539	74.20	8
SGD (momentum)	0.1	82.04	27	81.94	0.555	68.88	12
Adam	0.001	81.46	26	80.59	0.594	73.84	7
Adagrad	0.1	80.92	26	79.40	0.566	73.48	10
Nadam	0.001	80.90	28	79.33	0.587	71.62	10
RMSprop	0.001	77.66	28	76.84	0.736	66.52	12

Table 3. Each optimizer at its own best learning rate, sorted by best validation accuracy.

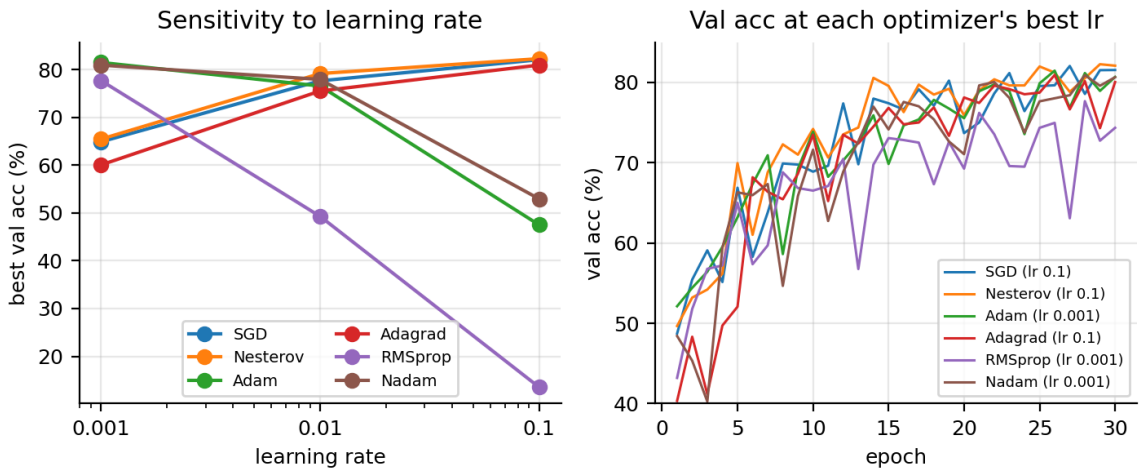


Figure 2. Left: best validation accuracy versus learning rate for each optimizer. Right: validation accuracy per epoch at each optimizer's best learning rate.

The best learning rate depends on the optimizer

The main finding is that the ranking of optimizers changes completely with the learning rate (Table 2). At lr = 0.001 the adaptive methods lead: Adam 81.46%, Nadam 80.90% and RMSprop 77.66%, while SGD (64.88%), Nesterov (65.44%) and Adagrad (60.04%) are still far from converged after 30 epochs (training loss 1.05, 1.04 and 1.23, versus 0.59 for Adam). At lr = 0.1 the order reverses: Nesterov (82.26%), SGD (82.04%) and Adagrad (80.92%) are best, whereas Adam drops to 47.50%, Nadam to 52.94% and RMSprop to 13.56%, which is chance level for 10 classes. Comparing optimizers at one shared learning rate would therefore give opposite conclusions. At each optimizer's own best learning rate (Table 3), five of the six finish within 1.4 points of each other (80.90% to 82.26% validation accuracy), which is about the noise level of a single-seed run on a 5,000-image validation set (Section 2c). These results therefore do not show that any one of the five is better in final accuracy; RMSprop (77.66%) is the only clear laggard.

SGD and Nesterov-SGD

Both improve steadily with the learning rate (SGD 64.88, 77.62, 82.04%; Nesterov 65.44, 79.14, 82.26%) and are still improving at the largest value tried, so their best learning rate may be at or above 0.1. BatchNorm probably helps make such a large step tolerable (we did not test this). At a small learning rate they, like Adagrad, are the slowest methods, because the step is proportional to the raw gradient with no per-parameter rescaling. Nesterov momentum evaluates the gradient at the look-ahead point. It gave a slightly lower final training loss than classical momentum at all three learning rates (by 0.009, 0.009 and 0.017) and reached 70% validation accuracy earlier at lr = 0.1 (epoch 8 vs. 12). Its validation gains (+0.56, +1.52, +0.22 points) and test differences (+0.28, +2.53, -0.29) are mixed and mostly within noise, so the honest summary is a small, consistent training-speed advantage and no reliable difference in final accuracy.

Adagrad

Adagrad divides each step by the square root of the sum of all past squared gradients, so its effective learning rate only decreases. With lr = 0.001 it is the slowest run of the sweep (60.04%, training loss 1.23 after 30 epochs) because the steps shrink before the network has learned much. A larger initial learning rate compensates: 75.50% at 0.01 and 80.92% at 0.1, where its training loss (0.566) is close to SGD's (0.556). Like SGD, its best learning rate is the largest tried. Its test accuracy at the best setting (79.40%) is 1.5 points below validation, a larger gap than the SGD variants (0.1 to 0.6 points).

RMSprop

RMSprop replaces Adagrad's sum with an exponential moving average, so the step size no longer decays, but it was the least robust optimizer here. At lr = 0.001 it learns quickly but its validation curve is very noisy (63.08% at epoch 27, then 77.66% at epoch 28) and it ends with the highest training loss of the runs that learned well (0.736). At lr = 0.01 training accuracy stalls near 50% and the validation loss spikes (up to 90.7 at epoch 19). At lr = 0.1 it does not learn at all (training and validation accuracy about 10%, training loss 2.6 to 2.9). Plausible contributors are the momentum term (0.9) used on top of the adaptive scaling, the absence of bias correction, and the fact that the weights change so fast that the BatchNorm running statistics used at evaluation no longer match the training-time batch statistics; we did not isolate which of these matters most.

Adam and Nadam

Both work best at lr = 0.001 (81.46% and 80.90%), degrade at 0.01 (76.54% and 77.92%) and fail at 0.1 (47.50% and 52.94%, training loss stuck around 1.3 to 1.4). Adam's normalisation makes each step roughly as large as the learning rate whatever the gradient scale, so a step of 0.1 per weight is larger than the initial weight scale of this network (standard deviation about 0.04 to 0.06) and prevents training from settling. Nadam adds Nesterov-style momentum to Adam and behaves very similarly: Adam is better at lr = 0.001 (+0.56 validation, +1.26 test), Nadam at 0.01 (+1.38 validation, +1.78 test) and at 0.1 (+5.44 validation), so Nadam was somewhat more tolerant of large learning rates, but there is no consistent winner at the learning rate that matters. Both start noisily (for example Nadam 40.26% at epoch 3 and Adam 58.62% at epoch 8) before stabilising.

Convergence speed

Adam was the fastest to reach 70% validation accuracy (epoch 7), followed by Nesterov (8), Adagrad and Nadam (10), and SGD and RMSprop (12) (Table 3). The validation curves are noisy, so the training loss after

30 epochs is a steadier measure of how far each optimizer got: Nesterov 0.539 < SGD 0.556 < Adagrad 0.566 < Nadam 0.587 < Adam 0.594 << RMSprop 0.736. In other words the adaptive methods (Adam, Nadam) are fastest in the first epochs at a small learning rate, but the SGD-type methods at a large learning rate catch up and finish with the lowest training loss, and the SGD variants also had the highest test accuracy (81.94% and 81.65% vs. 80.59% for Adam). This is the usual pattern reported for adaptive versus SGD-type optimizers, but with one seed and gaps of 1 to 2 points it should be read as suggestive, not established.

Caveats

- Only three learning rates, a factor of 10 apart, were tried. SGD, Nesterov and Adagrad peaked at the largest value (0.1), so their true optimum may be higher; the adaptive methods may do better somewhere between 3e-4 and 3e-3 (in Section 2c, Adam at batch size 128 reached 81.46% at both 1e-3 and 3e-3).
- Momentum (0.9) and weight decay (5e-5) were not varied, and each cell is a single seed.
- **Reproducibility check:** the Adam, lr = 0.001, batch size 128 run in this sweep is identical, epoch by epoch, to the same configuration run earlier for Section 2c (best 81.46% at epoch 26, test 80.59%), which confirms that the seeded pipeline is deterministic.

(c) Batch size, number of epochs and learning rate

Adam, ReLU, BatchNorm and weight decay 5e-5 were held fixed while batch size {64, 128, 256, 512} and learning rate {3e-4, 1e-3, 3e-3} were varied (12 runs, 30 epochs each). Table 4 lists the results; the number of optimizer updates per run is 21,120 / 10,560 / 5,280 / 2,640 for batch size 64 / 128 / 256 / 512.

Batch size	lr	Best val (%)	Best epoch	Test (%)	Train loss @30	Val @ epoch 10	First epoch val >= 70%
64	0.0003	79.28	30	78.40	0.689	65.78	8
64	**0.001**	**83.20**	**29**	**81.70**	**0.585**	**69.64**	**8**
64	0.003	81.96	27	81.07	0.616	71.54	7
128	0.0003	76.08	29	75.21	0.730	69.28	14
128	0.001	81.46	26	80.59	0.594	73.84	7
128	0.003	81.46	22	80.15	0.579	70.92	9
256	0.0003	72.84	30	72.14	0.785	64.12	19
256	0.001	78.58	30	77.95	0.625	69.24	12
256	0.003	81.96	30	80.48	0.575	72.92	7
512	0.0003	70.32	24	69.99	0.854	58.66	24
512	0.001	75.20	26	75.33	0.676	62.36	17
512	0.003	77.94	29	77.32	0.589	57.46	13

Table 4. Results of the batch size x learning rate grid (bold = best). 'Train loss @30' is measured on augmented batches during the epoch.

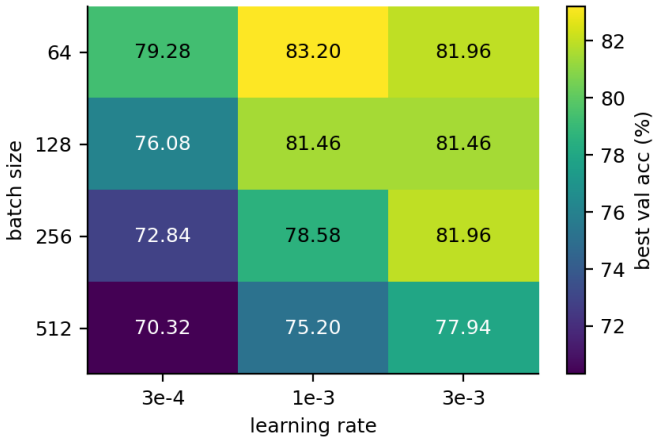


Figure 3. Best validation accuracy over the batch size x learning rate grid.

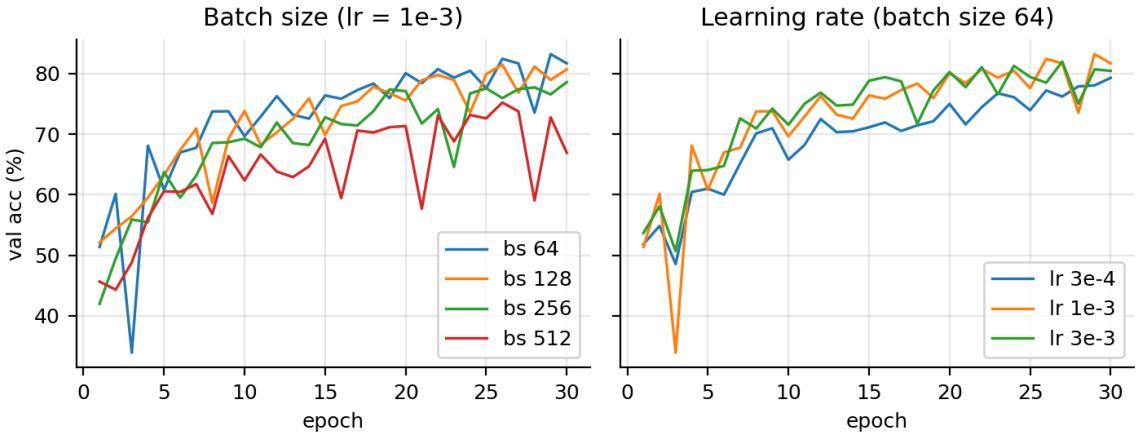


Figure 4. Validation accuracy per epoch. Left: effect of batch size at lr 1e-3. Right: effect of learning rate at batch size 64.

Batch size

At a fixed number of epochs, smaller batches won clearly. With lr = 1e-3, best validation accuracy fell from 83.20% (batch 64) to 81.46%, 78.58% and 75.20% for batch 128, 256 and 512. The training loss tells the same story (0.585, 0.594, 0.625, 0.676 after 30 epochs), so this is an *optimisation* effect, not a generalisation gap: a batch of 512 makes only 2,640 updates in 30 epochs versus 21,120 for batch 64, and with a fixed learning rate it simply has not travelled as far. Larger batches also gave smoother gradient estimates but, as Figure 4 (left) shows, did not make the validation curve less noisy at this learning rate, because the noise mainly comes from BatchNorm statistics and a constant learning rate. In wall-clock terms the larger batches gave no speed-up in our environment (about 4m45s per run at batch 64 versus 6m40s at batch 256 and 512), which suggests the input pipeline (CPU augmentation) rather than GPU compute was the bottleneck.

Learning rate

For batch size 64, lr = 1e-3 was best (83.20%), lr = 3e-3 slightly worse (81.96%), and lr = 3e-4 clearly slower (79.28% and still improving at epoch 30, final train loss 0.689 vs. 0.585). A learning rate that is too small converges slowly; one that is too large makes the validation curve oscillate more (Figure 4, right). The best learning rate grows with batch size: it is 1e-3 for batch 64, tied between 1e-3 and 3e-3 for batch 128, and 3e-3 for batch 256 and 512, the largest value we tried (for batch 512 the optimum may lie above 3e-3). This agrees with the common heuristic of increasing the learning rate with the batch size: each update averages more samples, so a larger step is affordable and compensates for the smaller number of updates. The best overall cell (batch 64, lr 1e-3) and the large-batch, high-lr cells (batch 256, lr 3e-3: 81.96%) end up within about 1 point of each other.

Number of epochs

A dedicated epoch sweep was run for the best 30-epoch setting (Adam, ReLU, lr 0.001, batch size 128): 10, 20 and 50 epochs, plus the 30-epoch run from the grid above (Table 5). The epoch sweep was run twice as two separate W&B sweeps (first, second) and gave identical results at every logged epoch, so Table 5 does not depend on which one is used.

Epochs	Best val (%)	Best epoch	Test (%)	Final train loss	Val gain vs. previous
10	73.84	10	74.46	0.832	-
20	77.82	18	77.86	0.678	+3.98
30	81.46	26	80.59	0.594	+3.64
50	84.66	50	83.48	0.506	+3.20

Table 5. Effect of the epoch budget (Adam, ReLU, lr 1e-3, batch size 128).

Validation accuracy rose with every increase in the budget: 73.84% (10 epochs), 77.82% (20), 81.46% (30) and 84.66% (50), and test accuracy followed (74.46%, 77.86%, 80.59%, 83.48%). The gain per additional epoch shrinks (about 0.40 points per epoch from 10 to 20, 0.36 from 20 to 30 and 0.16 from 30 to 50), but even at 50 epochs the training loss is still falling (0.506) and the best epoch is the last one, so training had not converged and no overfitting is visible.

Because the learning rate is constant and no scheduler is used, the first *k* epochs of a longer run are exactly what a *k*-epoch run produces with the same seed. The epoch sweep confirms this: the 10-, 20- and 50-epoch runs reproduce the first epochs of the 30-epoch run to the second decimal in every logged value. This lets us read off the effect of the epoch budget for all 12 grid runs from the best validation accuracy within the first 10, 20 and 30 epochs (Table 6; the batch 128, lr 1e-3 row equals Table 5).

Batch size	lr	10 epochs	20 epochs	30 epochs	Gain 20 to 30
64	0.0003	70.96	74.98	79.28	+4.30
64	0.001	73.76	80.04	83.20	+3.16
64	0.003	74.20	80.24	81.96	+1.72
128	0.0003	69.28	71.54	76.08	+4.54
128	0.001	73.84	77.82	81.46	+3.64

Batch size	lr	10 epochs	20 epochs	30 epochs	Gain 20 to 30
128	0.003	72.18	79.44	81.46	+2.02
256	0.0003	64.12	70.86	72.84	+1.98
256	0.001	69.24	77.36	78.58	+1.22
256	0.003	72.92	78.80	81.96	+3.16
512	0.0003	61.20	68.04	70.32	+2.28
512	0.001	66.38	71.32	75.20	+3.88
512	0.003	68.42	74.38	77.94	+3.56

Table 6. Best validation accuracy (%) for epoch budgets of 10, 20 and 30 for all 12 grid runs (read off from prefixes of the 30-epoch runs).

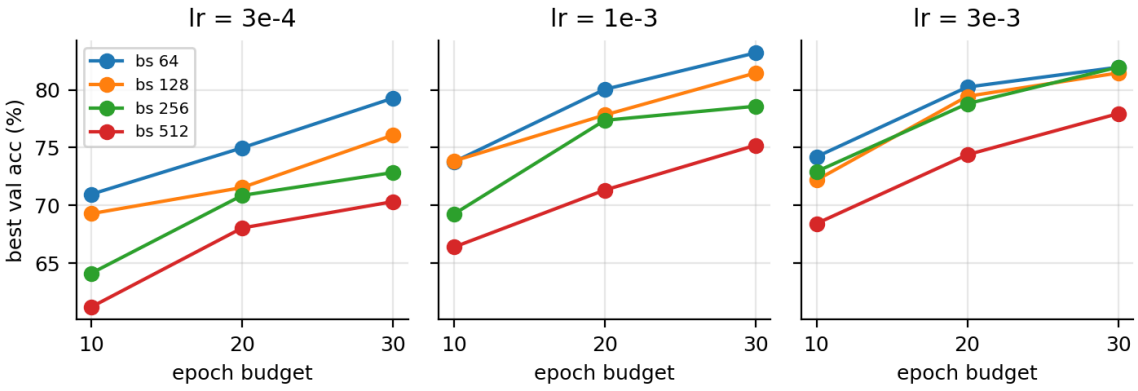


Figure 5. Best validation accuracy as a function of the epoch budget.

More epochs helped in every configuration: 10 to 20 epochs added 1 to 8 points and 20 to 30 epochs still added about 1 to 5 points in most cases. Seven of the 12 runs reached their best validation accuracy at epoch 29 or 30, and training loss was still falling in all of them, so none had converged. The slowest settings (large batch, small learning rate) benefit most from extra epochs (e.g. batch 128, lr 3e-4: 71.54% at 20 epochs vs. 76.08% at 30). Because no overfitting is visible, longer training (50 or more epochs) or a decaying learning-rate schedule would likely improve all of these results, and Table 5 confirms this for one configuration: extending training from 30 to 50 epochs gave a further 3.2 points of validation accuracy.

Reliability of these comparisons

Each cell is a single seed, and the validation set has 5,000 images, so the binomial standard error is about 0.55 points at 80% accuracy. In addition, validation accuracy fluctuates by several points from epoch to epoch, so 'best over epochs' is an optimistic estimate: across the 12 runs the test accuracy at the selected epoch was on average 0.83 points below the validation accuracy (range -0.13 to 1.50). Differences below roughly 1 point (for example 81.46% vs. 81.96%) should not be treated as meaningful; the large effects (batch size 64 vs. 512, lr 3e-4 vs. 1e-3) are well above this noise.

(d) Architecture and label smoothing (additional experiment)

The template network has 4 conv layers and 1 linear layer, i.e. 5 weight layers. To test whether capacity limits accuracy, three architectures were each trained with label smoothing 0 and 0.1 (6 runs; ReLU, Adam, lr 0.001, batch size 128, BatchNorm, 30 epochs, seed 42). **template**: conv64, conv64, pool, conv128, conv128, pool. **six_layer**: the template plus one conv256 block and a third pooling stage, giving 5 conv + 1 linear = 6 weight layers. **wide**: the template with doubled channel widths. Label smoothing 0.1 replaces the one-hot training target by 0.91 for the true class and 0.01 for each other class; it only changes the training loss, while validation loss and accuracy are always plain cross-entropy and top-1.

Architecture	Parameters	Label smoothing 0 (val / test)	Label smoothing 0.1 (val / test)
template	262,218	81.46 / 80.59	80.58 / 79.10
wide	1,040,522	83.68 / 83.44	83.38 / 82.04
six_layer	559,178	86.44 / 85.00	**87.22 / 86.29**

Table 7. Best validation / test accuracy (%) for each architecture and label-smoothing value (test accuracy at the best-validation epoch). Parameter counts are computed from the layer definitions (the template count matches the training logs).

Depth helped most. Adding one conv block raised the best validation accuracy from 81.46% to 86.44% (+4.98 points) with about 2.1 times the parameters, while doubling the width (about 4 times the parameters) gave +2.22 points (83.68%); test accuracy shows the same order (80.59% for template, 85.00% for six_layer, 83.44% for wide). For this network, an extra layer was therefore more parameter-efficient than extra width, plausibly because it adds a further resolution stage and a larger receptive field; this was not isolated (for example, a deeper network with fewer channels was not tried). The gap is visible throughout training and not only in the best epoch: in Figure 9 the six_layer runs lie above the others from about epoch 3 onwards, and at epoch 30 the group-mean validation loss is 0.434 for six_layer, 0.513 for wide and 0.592 for template (group means of the exported curves, which include both label-smoothing settings).

Label smoothing helped only the deepest network. For six_layer it improved the best validation accuracy by 0.78 points (86.44% to 87.22%) and the test accuracy by 1.29 points (85.00% to 86.29%); for wide and template it lowered validation accuracy by 0.30 and 0.88 points and test accuracy by 1.40 and 1.49 points. The direction agrees between validation and test for every architecture, but each cell is a single seed, so part of this pattern could be noise. It cannot be explained by parameter count alone, since the widest network (1.04 million parameters) did not benefit; a plausible but untested explanation is that the smoothed targets regularise the six_layer network usefully while slowing the fit of the smaller or shallower ones within 30 epochs.

The best run overall is **six_layer with label smoothing 0.1: 87.22% validation accuracy and 86.29% test accuracy**, both at the last epoch (30). This is 2.6 points (validation) and 2.8 points (test) above the best template result (84.66% and 83.48%, which needed 50 epochs), and since its best epoch is the last one it probably has headroom with a longer budget, as the template did in Section 2c; a 50-epoch six_layer run was not tried. This is the configuration reported in Q4.

Caveats: one seed per cell; and the two six_layer runs (and the two wide runs) share one W&B run name, so they must be told apart by their label_smoothing config value, not by name.

Question 3. Plots

Figures 6 to 9 are drawn from the data exported (CSV) from the corresponding panels of the W&B project, which contains 78 runs from all sweeps in this report (repeated configurations appear as overlapping lines or points). In the line plots, runs that share the same W&B run name are grouped: the line is the group mean and the shaded band the minimum and maximum across the runs in the group. The six groups are: template with 10, 20, 30 and 50 epochs (the 30-epoch group has 8 runs, including repeats of one configuration and one run with label smoothing 0.1) and six_layer and wide (30 epochs, 2 runs each: label smoothing 0 and 0.1). Training loss of label-smoothed runs includes the smoothing term, so it is not directly comparable with plain cross-entropy.

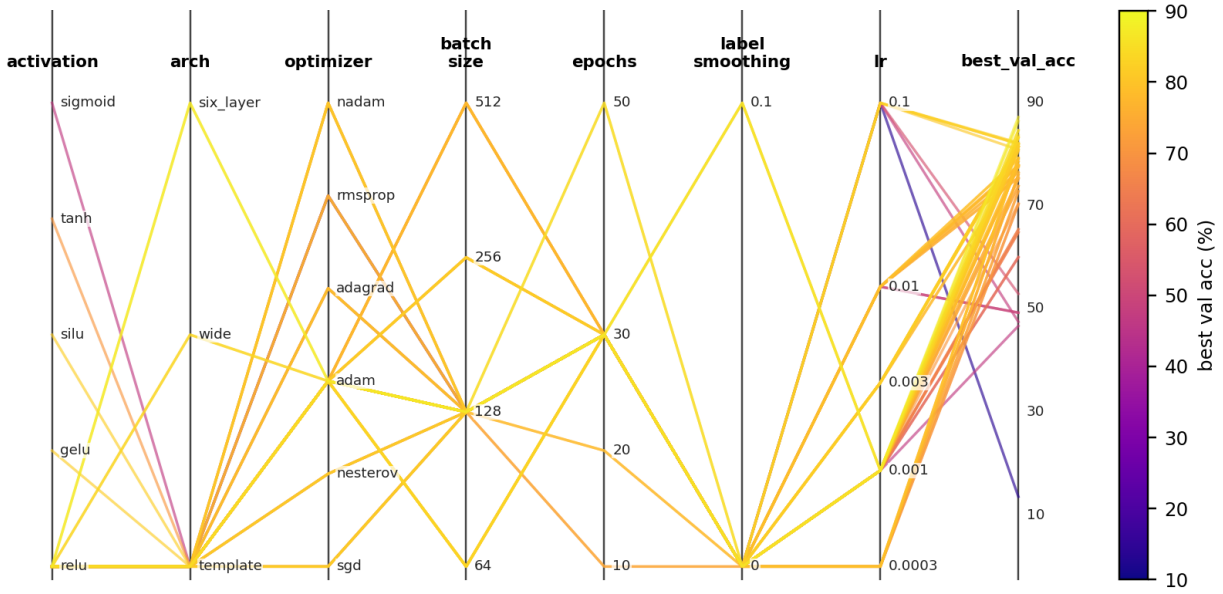


Figure 6. Parallel-coordinates plot: each line is one run, coloured by best validation accuracy (right-most axis).

- The lowest accuracies come from RMSprop at lr 0.1 (13.56%), Sigmoid (46.70%), Adam at lr 0.1 (47.50%) and RMSprop at lr 0.01 (49.28%); the best come from the deeper six_layer network (86.44% and 87.22%), followed by the 50-epoch template (84.66%).
- Among template runs, high accuracy is reached either with Adam (lr 0.001 to 0.003) or Nadam (lr 0.001), or with SGD / Nesterov / Adagrad at lr 0.1, as discussed in Section 2b.

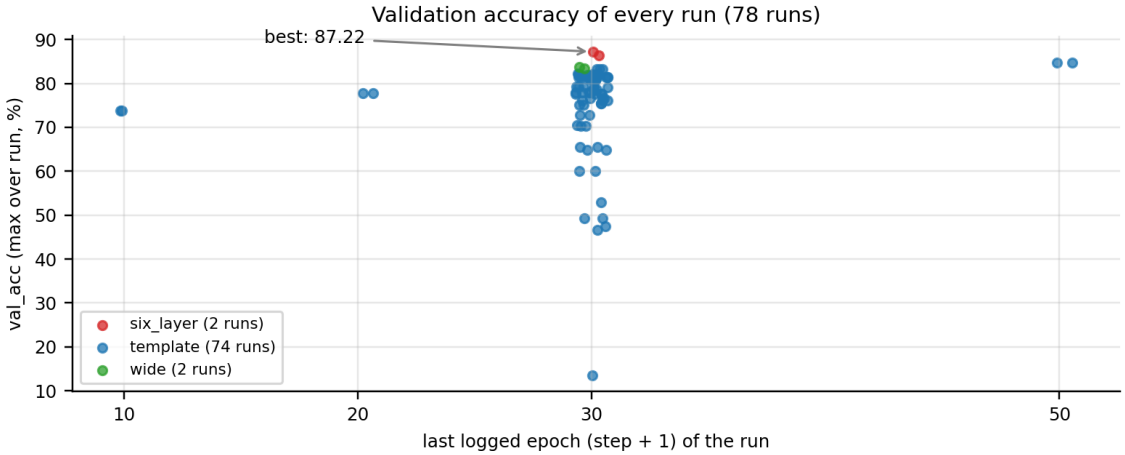


Figure 7. Validation accuracy of every run (maximum over the run) against the last logged epoch of the run.

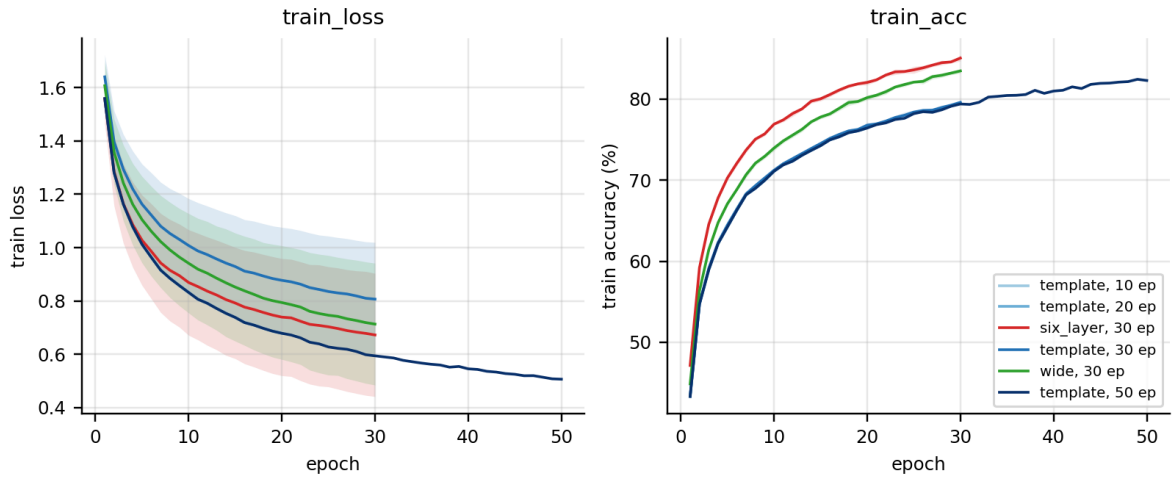


Figure 8. Training loss (left) and training accuracy (right) per epoch (group mean and min-max band).

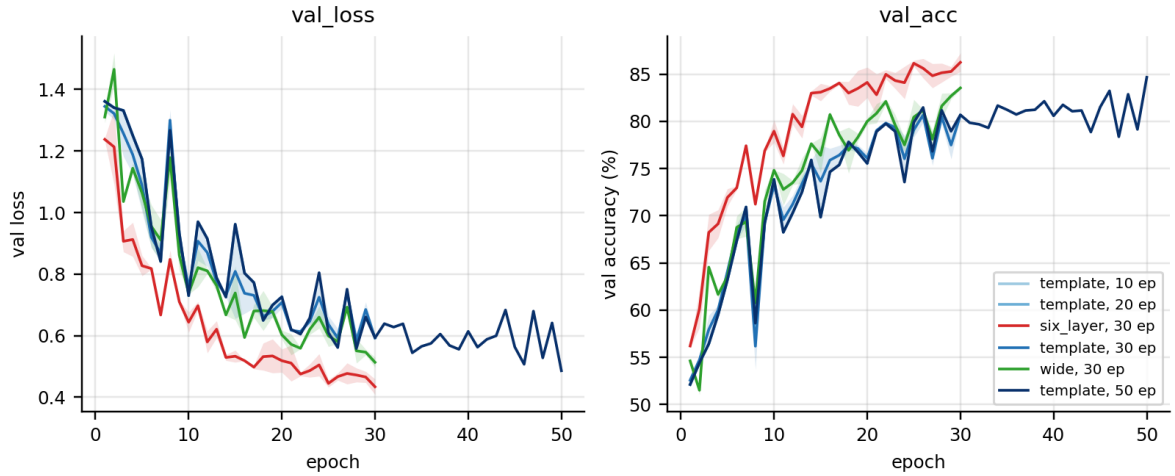


Figure 9. Validation loss (left) and validation accuracy (right) per epoch (group mean and min-max band).

- Validation accuracy rises with the epoch budget for the template network (10, 20, 30 and 50 epochs share the same first epochs, because the learning rate is constant and the runs are seeded), and the six_layer group is clearly highest and smoother than the others.
- The validation curves fluctuate by several points between epochs for all groups (Section 2c), which is why the best-epoch value is slightly optimistic.

Question 4. Final model performance

Parameter	Value
Architecture	six_layer (5 conv + 1 linear = 6 weight layers)
Label smoothing	0.1
Activation	ReLU
Optimizer	Adam
Learning rate	0.001
Batch size	128
Epochs	30
Weight decay	5e-5
BatchNorm / Cutout	True / True
Momentum	0.9 (not used by Adam)
Seed	42
Best validation accuracy	**87.22% (epoch 30)**
Test top-1 accuracy	**86.29%**

Reproduce with: `./reproduce_q4.sh` (or `bash reproduce_q4.sh`), which retrains this configuration from scratch without W&B, scores the new checkpoint with `evaluate.py`, copies it to `models/best_model.pt` and re-scores the committed model. The equivalent single command is `python train.py --arch six_layer --label_smoothing 0.1 --activation relu --optimizer adam --lr 0.001 --batch_size 128 --epochs 30 --weight_decay 0.00005 --batch_norm true --seed 42 --no_wandb`, followed by `python evaluate.py`. Runs are seeded (global seed, a dedicated GPU generator for shuffling and augmentation, fixed train/validation split) and use deterministic kernels.

This configuration was run twice (once in the architecture sweep and once as a standalone re-run with `--no_wandb`) and gave identical per-epoch results: best validation accuracy 87.22% at epoch 30, test accuracy 86.29% and test loss 0.4261; `evaluate.py` on the committed `models/best_model.pt` prints the same values. Other repeated configurations also reproduced exactly (the 50-epoch template run, 84.66% validation and 83.48% test, gave identical per-epoch values in two separate sweeps, and the 30-epoch template setting did so in seven runs, with a best validation accuracy of 81.46% every time). On different GPUs or library versions small differences (a few tenths of a point) are possible. The saved checkpoint is the best-validation epoch, which is the last epoch here.

Question 5. Reproducibility and repository

GitHub: https://github.com/srajiniranjith-collab/vgg6-cifar10

- **Code structure:** `train.py` (one training run, W&B logging, best-validation checkpoint), `evaluate.py` (scores a saved checkpoint on the test set), `src/data.py` (GPU data pipeline: crop, flip, normalise, Cutout, train/val/test batches), `src/model.py` (VGG6 with selectable activation, BatchNorm switch and architecture), `src/optimizers.py` (SGD, Nesterov, Adam, Adagrad, RMSprop, Nadam), `src/engine.py` (training and evaluation loops), `src/utils.py` (seeding), `sweeps/*.yaml` (W&B sweep definitions), `reproduce_q4.sh` (one-command reproduction of the Q4 model), `tests/smoke_test.py`.
- **README:** exact commands for single runs, every sweep and the final verification; environment and dependency versions; seed configuration (global seed 42, seeded GPU generator for shuffling and augmentation, fixed 45k/5k split, deterministic cuDNN and a deterministic global-average-pooling implementation).
- **Trained model:** the final model checkpoint is committed as `models/best_model.pt` (weights of the best-validation epoch of the Q4 configuration).